

POCKET SMART AI Your Smart Budget & Recommendation Assistant COMPLETE.

PROJECT REPORT

ST. GEORGE'S ARTS AND SCIENCE COLLEGE.(UNM1727)
TN SKILLS | GENERATIVE AI



Team ID : SWTID-2026-2087 / 6ab7806b6e62b46dee66a620

Team Size : 5

Team Leader : MONISH B

Team member : JAGADEESHWARAN S

Team member : SHANNON CADRIC JACOB

Team member : PREETHIKA B

Team member : NARESH S

Document Control & Report Scope

This report follows the 20-part project structure supplied in the user-provided material. It is written as a submission-ready document while keeping unsupported implementation details explicitly unclaimed.

Field	Detail
Project	Pocket Smart AI
Report type	Complete project report
Coverage	Requirements → Design → Development → Testing → Deployment
Primary application concept	Fixed-budget multi-category planning and recommendation
Source note	Uploaded project material and supplied report format

Executive Summary

- Pocket Smart AI is a web-based budgeting and recommendation assistant intended to support users who must allocate a fixed total amount across several related purchase requirements. The project emphasizes planning before spending, rather than limiting the workflow to historical expense recording.
- The report documents the project from problem definition and requirements through system design, environment setup, core development, integration, testing, deployment, results, limitations, and future enhancement. The application screenshots supplied with the project are treated as visual evidence of the user-facing application flow.

Table of Contents

SNO	INDEX	PAGE
1.	PROJECT TITLED	
2.	PROJECT DESCRIPTION	
3.	APPLICATION SCENARIOS / USE CASES	
4.	TECHNICAL ARCHITECTURE OVERVIEW	
5.	PREREQUISITES	
6.	PRIOR KNOWLEDGE REQUIRED	
7.	PROJECT OBJECTIVES	
8.	SYSTEM WORKFLOW	
9	MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN	
10.	MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION	
11.	MILESTONE 3: CORE SYSTEM DEVELOPMENT	
12.	MILESTONE 4: INTEGRATION & OPTIMIZATION	
13.	MILESTONE 5: TESTING & VALIDATION	
14.	DEPLOYMENT	
15	PROJECT STRUCTURE	
16.	RESULTS	
17.	ADVANTAGES & LIMITATIONS	
18.	FUTURE ENHANCEMENTS	
19.	CONCLUSION	
20.	APPENDIX	

1. PROJECT TITLE

- **Project Title:** POCKET SMART AI – Your Smart Budget & Recommendation Assistant
- **Project Type:** Web-based intelligent budgeting and recommendation application.
- **Purpose:** To help users plan a fixed budget across multiple interdependent purchase categories and receive structured, forward-looking purchasing guidance.

2. PROJECT DESCRIPTION

- **System Overview:** Pocket Smart AI is designed as a digital budgeting and recommendation assistant. Instead of limiting the user to recording expenses after they occur, the system focuses on planning: the user provides a budget and purchasing needs, and the application organizes the available amount across relevant categories and presents decision-support guidance.
- **Problem Domain:** The project addresses personal budgeting and multi-item purchase planning. A common challenge is deciding how much of a fixed amount should be allocated to several related requirements while maintaining the overall budget.
- **Core Technology Used:** The supplied material establishes a browser-based application with user-facing screens for entry and access, together with a dashboard-oriented application flow. The exact framework, runtime version, database product, and AI/API provider are not exposed by the supplied report package, so this report does not invent those details.
- **Key Capabilities:** User access flow; budget and requirement input; structured budget allocation; recommendation-oriented output; dashboard presentation; validation and error handling; deployment as a web application.
- **Target Users:** Students, individual consumers, households, event planners, home-decor planners, and other users who need to distribute a fixed amount across several connected purchase requirements.
- **Real-world Relevance:** The system can support forward-looking purchase decisions where several categories compete for the same total budget. It is particularly relevant when users need a practical allocation plan rather than only historical expense records.

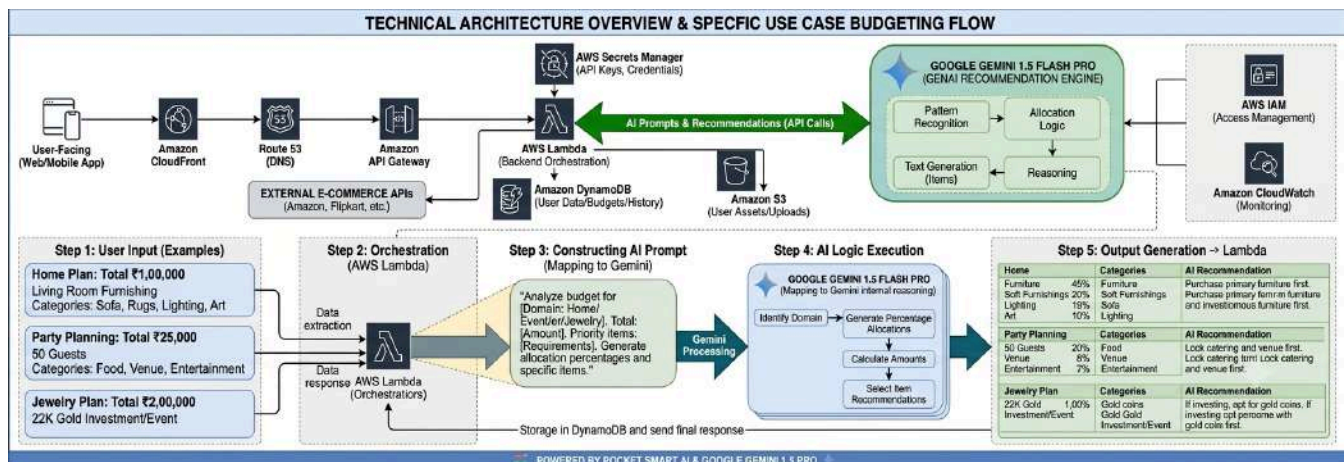
3. APPLICATION SCENARIOS / USE CASES

- **Scenario 1 – End-user purchase planning:** A user has a fixed amount and several items to purchase. The user enters the overall budget and requirements. Pocket Smart AI processes the inputs and presents an organized allocation and recommendation view.
- **Scenario 2 – Home-decor planning:** A user plans a room or home-decor purchase involving several categories such as furniture, lighting, accessories, and other requirements. The system can be used to structure the available budget before purchasing.

- **Scenario 3 – Event planning:** An event planner or student organizing an event can distribute a fixed amount among multiple categories. The application can act as a planning assistant before commitments are made.
- **Scenario 4 – Educational usage:** The application can be demonstrated in a classroom or project laboratory to teach requirements analysis, UI design, input validation, application workflow, budgeting logic, and deployment practices.

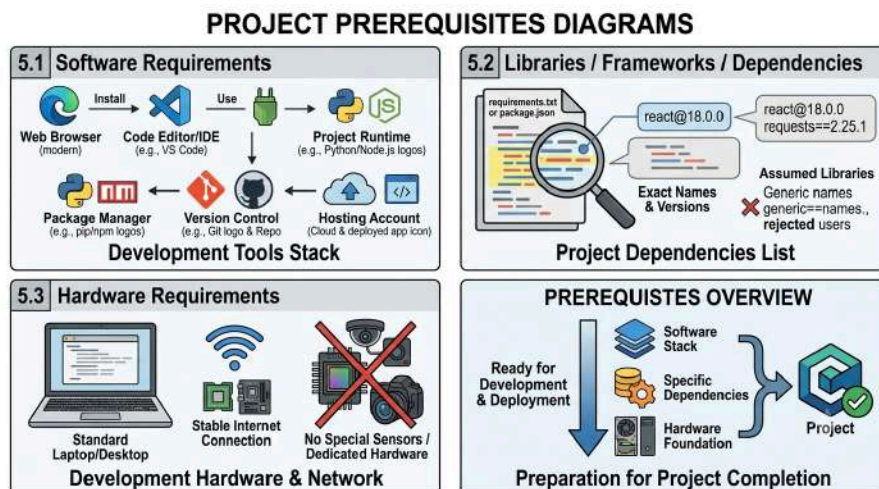
4. TECHNICAL ARCHITECTURE OVERVIEW

- **Architecture:** The application follows a layered web-application concept: user/browser interaction, presentation layer, application logic, data/state handling, and optional external services. The precise implementation technologies should be filled from the final project source code if a framework-specific submission is required.
- **Frontend Layer:** Provides the user-facing pages and forms for registration/login, input collection, navigation, and result/dashboard presentation.
- **Backend / Application Layer:** Responsible for validating requests, applying budgeting and recommendation logic, coordinating application operations, and preparing output for the interface.
- **Database Layer:** If persistent storage is enabled, it stores user/application data according to the selected implementation. The supplied material does not establish a specific database product, so no product name is asserted here.
- **External APIs / Services:** External AI, recommendation, authentication, or hosting services may be integrated depending on the final implementation. No specific API provider is claimed without source evidence.
- **Deployment Environment:** The project is intended to run as a web application accessible through a browser. Local development and a hosted production environment are separate deployment stages.

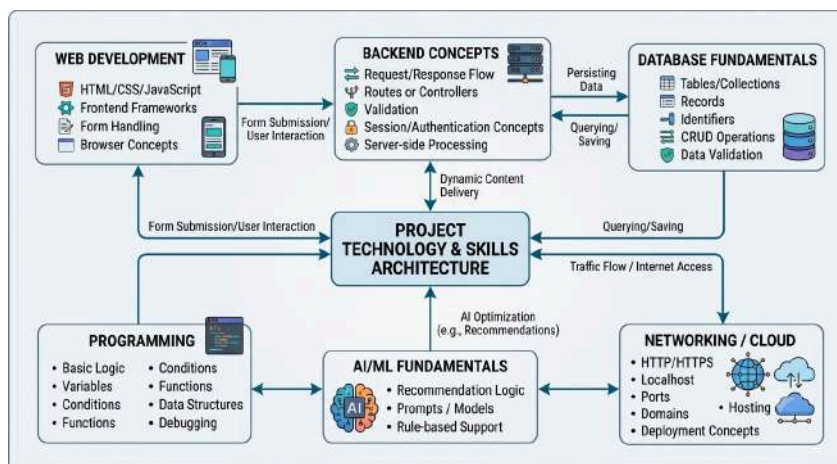


5. PREREQUISITES

- **5.1 Software Requirements:** A modern web browser; code editor/IDE; the runtime required by the selected project implementation; package/dependency manager where applicable; version-control tooling for the project repository; and a hosting account if the application is deployed online.
- **5.2 Libraries / Frameworks / Dependencies:** The exact dependency list must match the project's actual dependency/configuration files. For the final submission, the project team should copy the exact package names and versions from its requirements/configuration file rather than substituting assumed libraries.
- **5.3 Hardware Requirements:** A standard laptop or desktop capable of running a browser and the project's development runtime. A stable internet connection is recommended when the application uses hosted services. No special sensors or dedicated hardware are required by the project concept.



6. PRIOR KNOWLEDGE REQUIRED

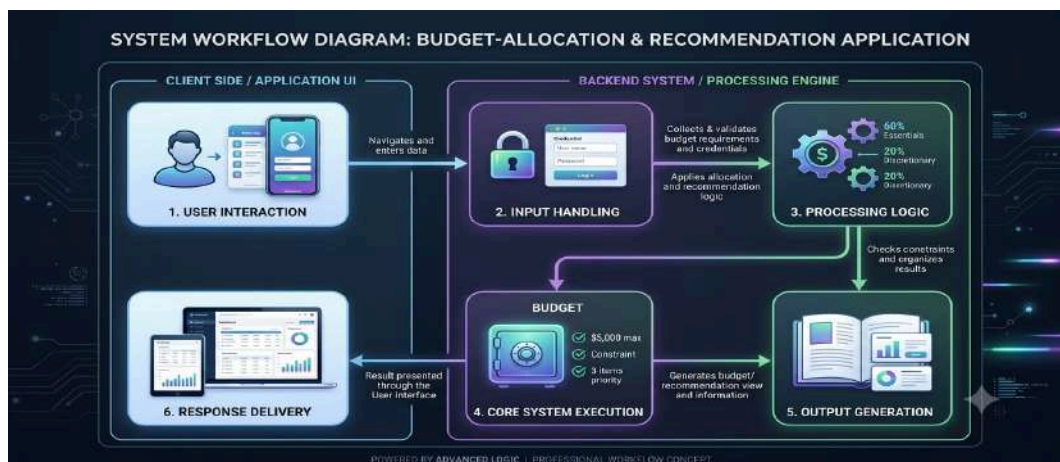


- **Programming:** Basic programming logic, variables, conditions, functions, data structures, and debugging.
- **Web Development:** HTML/CSS/JavaScript or the equivalent frontend technology used by the project; basic form handling and browser concepts.
- **Backend Concepts:** Request/response flow, routes or controllers, validation, session/authentication concepts, and server-side processing where applicable.
- **Database Fundamentals:** Tables/collections, records, identifiers, CRUD operations, and data validation if persistent storage is used.
- **Networking / Cloud:** Basic HTTP/HTTPS, localhost, ports, domains, hosting, and deployment concepts.
- **AI/ML Fundamentals:** Basic understanding of recommendation logic, prompts/models or rule-based decision support if an AI service is used.

7. PROJECT OBJECTIVES

- **Technical Objectives:** Develop a usable web application for budget planning; implement structured input and validation; organize business logic into understandable components; provide clear recommendation-oriented output; and maintain a clean project structure.
- **Performance Objectives:** Provide responsive user interaction, avoid unnecessary processing, validate input early, and keep the recommendation flow understandable and efficient.
- **Deployment Objectives:** Make the application reproducible in a local development environment and suitable for deployment to a web hosting environment.
- **Learning Outcomes:** Gain practical experience in requirement analysis, UI development, application logic, testing, deployment, documentation, and project presentation.

8. SYSTEM WORKFLOW



1. **User interaction:** The user opens the application and navigates to the required access/input screen.
2. **Input handling:** The application collects user credentials and/or budgeting requirements and validates mandatory fields.
3. **Processing logic:** The application receives the validated values and applies its budget-allocation and recommendation logic.
4. **Core system execution:** The system checks constraints such as the total available budget and the set of selected requirements, then prepares an organized result.
5. **Output generation:** The application generates the budget/recommendation view and supporting information for the user.
6. **Response delivery:** The result is returned to the browser and presented through the application's dashboard or result interface.

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

- **9.1 Problem Definition:** Users planning multiple purchases often need to make several connected decisions under one fixed budget. A useful system should support planning before spending, preserve the total budget constraint, and make the result easy to understand.
- **9.2 Functional Requirements:** FR-01: Provide an application entry point. FR-02: Support user access/registration flow where implemented. FR-03: Accept budget and requirement inputs. FR-04: Validate input values. FR-05: Process the budget allocation. FR-06: Generate recommendations or guidance. FR-07: Present results through the interface. FR-08: Handle invalid or incomplete input without crashing.
- **9.3 Non-Functional Requirements:** Usability, responsiveness, maintainability, reliability, basic security hygiene, predictable validation, readable output, and deployment reproducibility.
- **9.4 System Design Decisions:** Use a modular flow separating presentation, input handling, business logic, and data/state concerns. Keep user-facing output simple enough for a non-technical consumer while keeping the internal logic maintainable.
- **9.5 Technology Stack Selection Justification:** A browser-based architecture was selected because the problem is user-interaction heavy and benefits from easy access across devices. The final implementation should document the exact frontend, backend, storage, AI/API, and hosting technologies used in the repository.

10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

- **10.1 Development Environment Setup:** Install the project's runtime and package manager, configure the IDE/editor, clone or open the project repository, and verify that the application starts successfully in the local environment.
- **10.2 Dependency Installation:** Install dependencies from the project's actual configuration file. Confirm that all required packages resolve successfully and that the application starts without missing-module errors.
- **10.3 Project Structure Creation:** Recommended logical structure: project_root/src, project_root/config, project_root/static, project_root/templates, project_root/database (if applicable), project_root/tests, main application file, and requirements/configuration file.
- **10.4 Configuration Setup:** Keep environment-specific values outside source code where possible. Configure local ports, service credentials, database connection values, and API keys through secure environment configuration rather than hard-coding secrets.

11. MILESTONE 3: CORE SYSTEM DEVELOPMENT

- **11.1 Feature 1 – User Access:** The project screenshots show an application access flow containing registration/login-oriented screens. This feature provides the entry point for users and establishes the basic user journey.
- **11.2 Feature 2 – Budget & Requirement Processing:** The core project concept is fixed-budget planning across multiple interdependent categories. The application accepts planning inputs, checks the total budget constraint, and prepares an allocation-oriented result.
- **11.3 Feature 3 – Recommendation / Dashboard Output:** The system presents processed information in a dashboard-oriented interface so the user can review the resulting plan. The recommendation layer should be documented according to the exact implementation: rules, formulas, AI model, or external service, as applicable.
- **Database / API / Model Architecture:** If the final implementation uses persistent data, APIs, or an AI model, the repository should be treated as the authoritative source for the exact schema, endpoints, model name, and configuration. These details are intentionally not invented in this report.

12. MILESTONE 4: INTEGRATION & OPTIMIZATION

- **12.1 Component Integration:** Integrate the user interface with the application's processing logic so that inputs move through validation, calculation/recommendation, and result rendering as one continuous flow.

- **12.2 Performance Optimization:** Validate early, avoid repeated processing, minimize unnecessary network requests, keep result generation deterministic where possible, and optimize static assets for faster loading.
- **12.3 Security Enhancements:** Validate all user input, protect authentication data, avoid exposing secrets in source code, use HTTPS in production, and apply least-privilege access to any persistent storage or external services.
- **12.4 Error Handling & Validation:** Handle missing fields, invalid numeric values, impossible budget combinations, service failures, and unexpected application errors with user-friendly messages and safe fallbacks.

13. MILESTONE 5: TESTING & VALIDATION

- **13.1 Test Cases:** The test plan below covers representative functional behavior. Final status should be updated after executing the test suite on the submitted build.
- **13.2 Unit Testing:** Test individual validation, calculation, allocation, recommendation, and formatting functions independently.
- **13.3 Integration Testing:** Verify that browser input reaches the correct application logic and that processed results are returned and displayed correctly.
- **13.4 User Acceptance Testing:** Ask representative users to complete the full planning flow and confirm that the screens, terminology, calculations, and recommendations are understandable.
- **13.5 Performance Metrics:** For a web application, record response time, page-load behavior, concurrent-load observations, and security-validation results. If a machine-learning model is actually used, record accuracy/precision/recall/F1 only when those metrics are meaningful and measured.

13.1 Representative Test Case Matrix

Test Case ID	Input	Expected Output	Status
TC-01	Open application	Landing/access screen loads	To be executed
TC-02	Valid registration data	Account registration completes	To be executed
TC-03	Invalid/empty required field	Validation message shown	To be executed
TC-04	Valid login credentials	User enters application	To be executed
TC-05	Budget + valid requirements	Allocation/recommendation generated	To be executed
TC-06	Budget exceeded by	System prevents invalid	To be executed

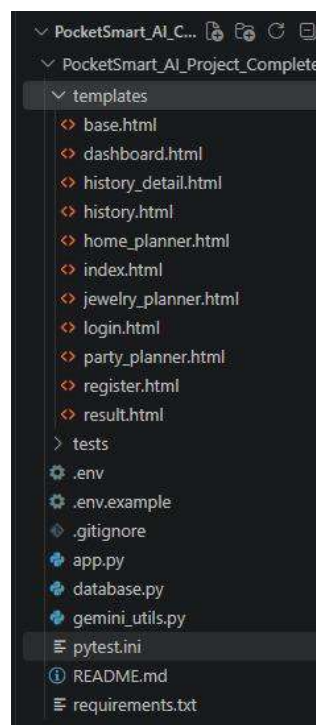
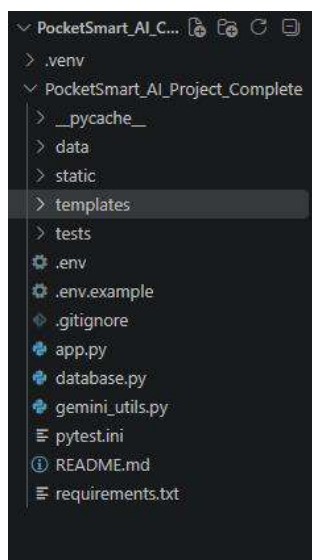
Test Case ID	Input	Expected Output	Status
	inputs	total or explains constraint	
TC-07	Invalid numeric input	Input rejected safely	To be executed
TC-08	Result/dashboard request	Processed result rendered	To be executed
TC-09	Unexpected/service failure	Friendly error/fallback shown	To be executed
TC-10	Production URL	Application accessible over intended deployment URL	To be executed

14. DEPLOYMENT

- **14.1 Deployment Architecture:** Browser → hosted web application → application logic → optional persistent storage and/or external services.
- **14.2 Hosting Platform:** The exact hosting provider is not established by the supplied source material. The final report should state the provider actually used and include the live deployment URL.
- **14.3 Deployment Steps:** Prepare production configuration; install dependencies; configure environment variables; initialize storage if required; run application checks; deploy to the hosting platform; verify the public URL; and perform smoke tests.
- **14.4 Production Considerations:** Use HTTPS, secure secrets, production logging, error monitoring, backup strategy where data is persistent, input validation, access control, and a rollback/redeployment procedure.

15. PROJECT STRUCTURE

Recommended Structure:



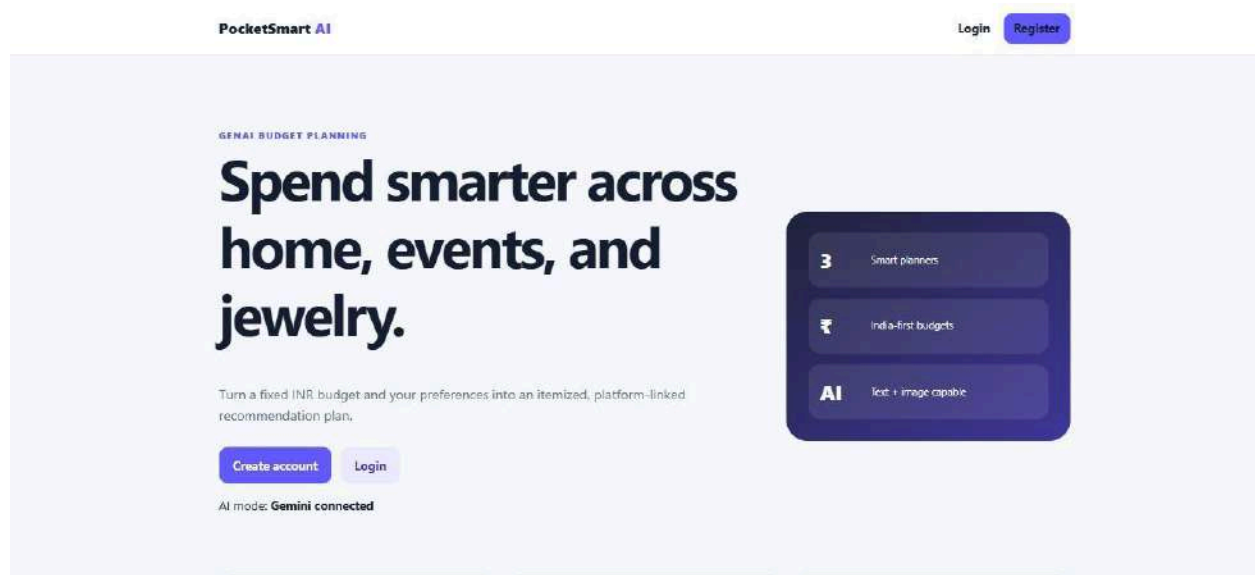
Folder Explanation:

- config/: **environment/configuration definitions.**
- static/: **CSS, JavaScript, images, and other static assets.**
- templates/: **HTML or presentation templates.**
- database/: **schema, seed data, migrations, or storage helpers if applicable.**
- tests/: **automated/manual test assets.**
- main_application_file: **application entry point.**
- requirements.txt or equivalent: **dependency definition.**

16. RESULTS

1. **System Output:** The completed system is intended to provide an accessible budgeting and recommendation workflow, moving from user access and input to an organized planning result.
2. **Performance Evaluation:** Performance should be reported from actual measurements on the final deployment. This report does not fabricate response-time or load-test numbers that were not supplied.
3. **Screenshots:** The submission includes application screenshots treated as visual evidence of the user-facing application flow.

Screenshot Evidence Register



SCREENSHOT 1: Landing Page

PocketSmart AI

[Login](#) [Register](#)

GET STARTED

Create account

Full name

Username

Email

Password

Create account

PocketSmart AI - Recommendations and prices are estimates; verify live prices before purchasing.

SCREENSHOT 2: Create Account

PocketSmart AI

[Login](#) [Register](#)

WELCOME BACK

Login

Username

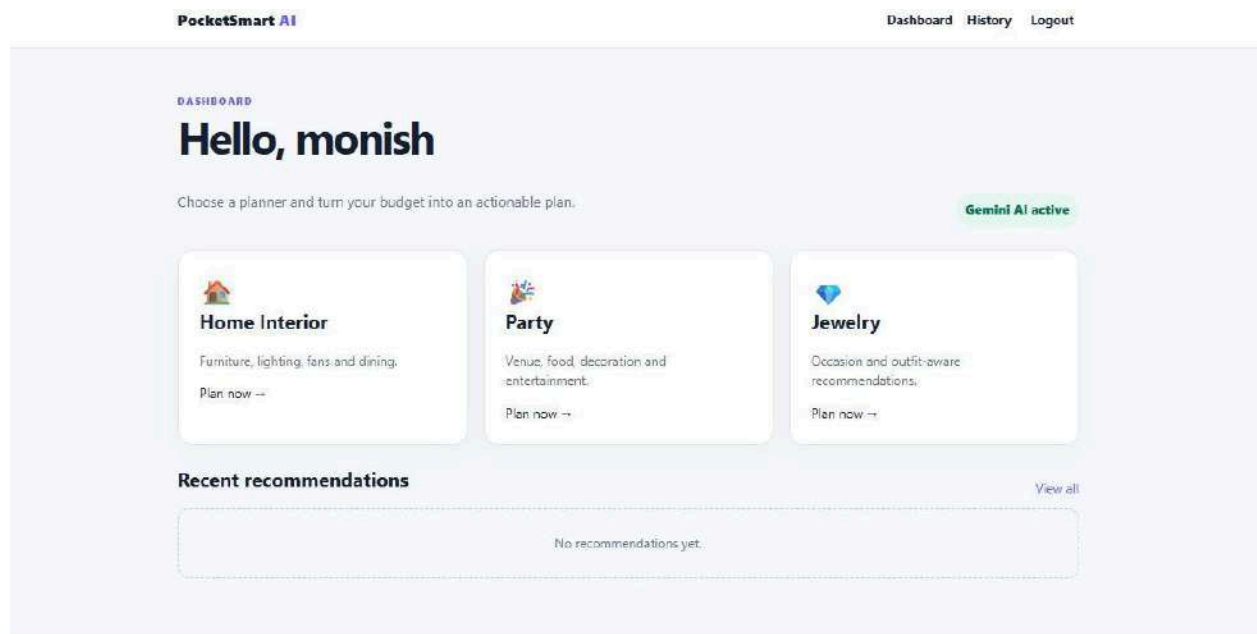
Password

Login

New here? [Create an account.](#)

PocketSmart AI - Recommendations and prices are estimates; verify live prices before purchasing.

SCREENSHOT 3: Login



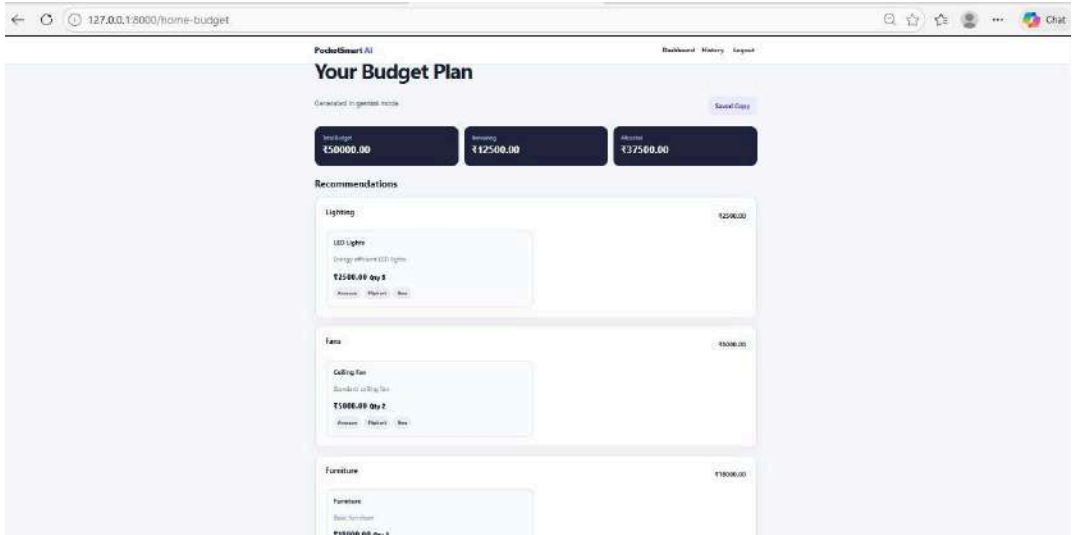
PocketSmart AI - Recommendations and prices are estimates; verify live prices before purchasing.

SCREENSHOT 4: Dashboard

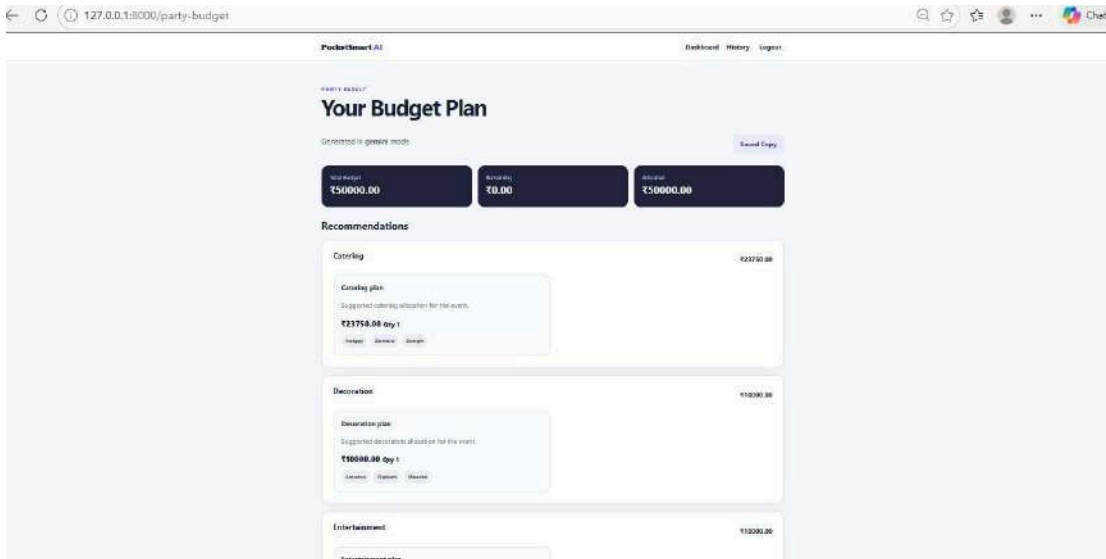
The screenshot shows the "Plan a home budget" form in the PocketSmart AI application. The navigation bar at the top has "PocketSmart AI" on the left and "Dashboard", "History", and "Logout" on the right. Below the navigation bar, the word "HOME INTERIOR" is in small letters, followed by the title "Plan a home budget". A sub-header says "The generated plan is capped at your budget." Below this, there's a white form with rounded corners. The form has several input fields: "Total budget (₹)" with the value "50000", "Lights" with the value "5", "Ceiling fans" with the value "2", "Furniture pieces" with the value "3", and "Dining tables" with the value "1". Below these fields, there's a "Rooms" section with three checkboxes: "Living room" (checked), "Kitchen" (unchecked), and "Bedroom" (checked). Below the rooms section, there's an "Additional requirements" section with a text input field. At the bottom of the form, there's a blue button labeled "Generate home plan".

PocketSmart AI - Recommendations and prices are estimates; verify live prices before purchasing.

SCREENSHOT 5: Plan a Home Budget Form



SCREENSHOT 6: Home Budget Result



SCREENSHOT 7: Party Budget Result

PocketSmart AI

DashboardHistoryLogout

JEWELRY SETTINGS

Find jewelry within budget

Optionally upload an outfit for Gemini visual analysis.

Total budget (₹)

15000

Occasion

Birthday

Style preferences

minimal gold tone

Outfit image (optional)

Choose File 211fa25c-6134-4d07-b6db-11e93ed71f64.jpg

Generate jewelry plan

PocketSmart AI

DashboardHistoryLogout

JEWELRY RESULT

Your Budget Plan

Generated in gemini mode

Saved Copy

Total Budget

₹15000.00

Remaining

₹2700.00

Allocated

₹12300.00

Outfit Analysis

Color

Style

Formality

Image supplied

Dense mode cannot reliably infer style

Configure Gemini for visual outfit analysis

Jewelry Recommendations

Pendant necklace

Versatile option suitable for a Birthday occasion.

₹5400.00

AmethystFluoriteBluestoneTurquoiseCarnelianMalachite

View details

Earrings

Versatile option suitable for a Birthday occasion.

₹3900.00

AmethystFluoriteBluestoneTurquoiseCarnelianMalachite

View details

Bracelet

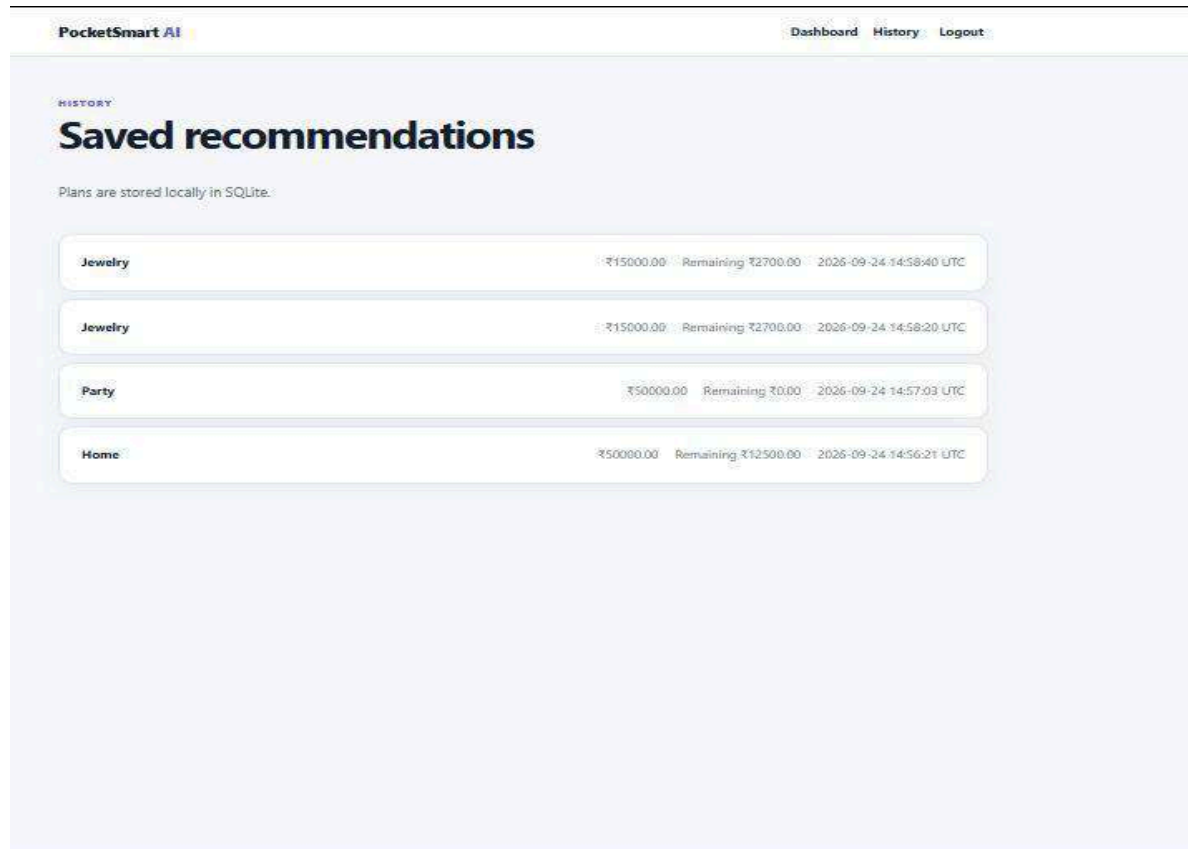
Versatile option suitable for a Birthday occasion.

₹3000.00

AmethystFluoriteBluestoneTurquoiseCarnelianMalachite

View details

SCREENSHOT 8: Find Jewelry Within Budget Form



SCREENSHOT 9: Localhost Browser History

17. ADVANTAGES & LIMITATIONS

1. Advantages:

- Forward-Looking Planning: Focuses on proactive budgeting rather than only historical expense tracking.
- Complex Decision Support: Highly useful for fixed-budget, multi-category purchasing decisions.
- Accessibility: Simple, browser-based access requiring no complex installations.
- Actionable Insights: Delivers recommendation-oriented output rather than just raw numbers.
- Educational Value: Serves as an excellent practical use case for full-stack project development and architecture learning.
- Scalability: Strong potential for future AI and third-party API service integration.

2. Limitations:

- Recommendation quality depends on the quality of user inputs and the implemented logic/model; exact technology and service dependencies may affect portability; external AI/API usage can introduce cost, latency, or availability concerns; real-world purchasing prices can change; the supplied project documentation does not expose measured production benchmarks.

18. FUTURE ENHANCEMENTS

- **Scalability:** Introduce stronger modular architecture, caching, background processing, and scalable data storage when user volume increases.
- **Feature Expansion:** Add category templates, recurring budgets, purchase history, saved plans, comparison views, alerts, and exportable budget reports.
- **Cloud Integration:** Integrate managed authentication, cloud databases, secure secret management, monitoring, and automated deployment.
- **Mobile Integration:** Provide a responsive PWA or dedicated mobile application for faster on-the-go planning.
- **Automation:** Add price-aware recommendations, automatic category suggestions, reminders, and personalized planning workflows where reliable data sources are available.

19. CONCLUSION

- **Summary of Implementation:** Pocket Smart AI is a project focused on practical budget planning and recommendation support. Its central idea is to help a user distribute a fixed amount across connected requirements and review the resulting plan through a web interface.
- **Technical Achievements:** The project addresses user access, structured input, validation, budgeting logic, recommendation-oriented output, interface integration, testing, and deployment as a coherent application lifecycle.
- **Practical Impact:** The concept can reduce the friction involved in planning multi-item purchases by turning a broad budget into a structured decision-support workflow. Its design also provides a strong base for later personalization, cloud integration, and AI-assisted recommendations.

20. APPENDIX

- **Source Code:**
https://drive.google.com/drive/folders/1ixWXNh8VjUMNjh6GnjAcxU0NtqRPP3_z
- **Live Demo Link:**
https://drive.google.com/file/d/127hGGCOfns8FGi5IUHsy_MfivP1LTfdU/view?usp=drivesdk
- **Documents drive folders:**
<https://drive.google.com/drive/folders/1uGTrGr2nbU5G6e1khQycmfN1kxoCQuSC>
